

Q1.

當訂閱的 topic 為 `/a/test` 時，能不能接收到 topic 為 `a/test` 的訊息？為什麼？

(hint : 比較兩個 topic 的層數)

Ans : 不能。

MQTT 的 topic 以「/」作為階層分隔符號，開頭的「/」本身也會切出一個「空字串」的層級。因此 `/a/test` 共有 3 層（`""`、`"a"`、`"test"`），而 `a/test` 只有 2 層（`"a"`、`"test"`）。訂閱的 topic 沒有使用萬用字元（`+`、`#`）時，必須每一層都完全相同才算匹配；兩者層數不同、第一層也不同（`"" ≠ "a"`），所以訂閱 `/a/test` 的 subscriber 收不到發佈到 `a/test` 的訊息。

實驗驗證：Terminal 1 訂閱 `/a/test`，Terminal 2 先 publish 到 `a/test` (`test_q1`)，subscriber 沒有任何輸出；再 publish 到 `/a/test` (`test_q1_compare`) 作對照，subscriber 才收到訊息。



圖 1 Q1 測試：只有 `/a/test` 的訊息被收到，`a/test` 的 `test_q1` 沒有出現

Q2.

Msg format : Place/Floor/Sensor Type/Time/ 共 36 種

- **Place** : `house1`、`house2`
- **Floor** : `firstfloor`、`secondfloor`、`roof`
- **Sensor Type** : `temp`、`humid`、`brightness`
- **Time** : `day`、`night`

若只想收到以下內容，那麼 subscriber 的 topic 分別要定為什麼？

1. 所有 `roof` 在 `day` 的 `brightness`
2. `house1 firstfloor` 的所有內容
3. `house2` 在 `night` 的所有內容
4. 所有 `roof` 的所有內容

Ans :

(1) 所有 `roof` 在 `day` 的 `brightness` : `+/roof/brightness/day` → 實測收到 2 則

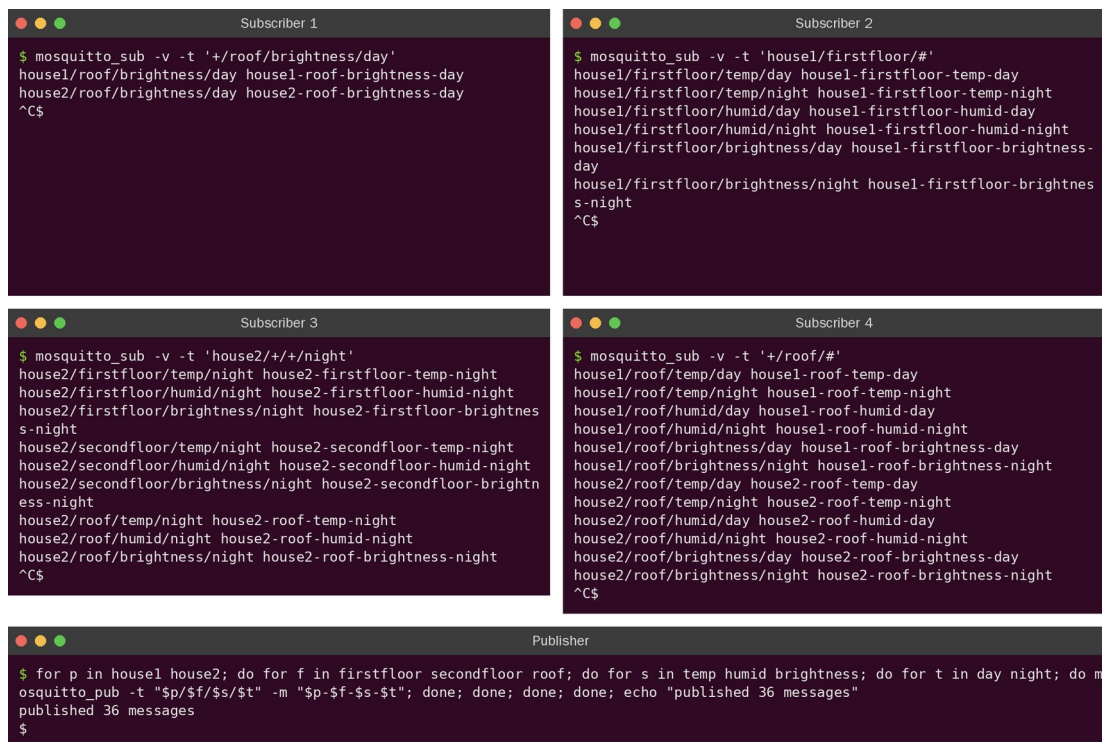
(2) `house1 firstfloor` 的所有內容 : `house1/firstfloor/#` (亦可寫成 `house1/firstfloor/+/+`) → 實測收到 6 則

(3) `house2` 在 `night` 的所有內容 : `house2/+/+/night` → 實測收到 9 則

(4) 所有 roof 的所有內容：**+/**roof**/#** (亦可寫成 **+/**roof**/+/+**) → 實測收到 12 則

說明：「+」只能代表「單一」階層，可放在任何位置；「#」代表其後的任意多個階層，只能放在最後一層。第 (1)(3) 題要固定某幾層、其他單層任意，所以用「+」；第 (2)(4) 題要某層以下的所有內容，所以在結尾用「#」。將 $2 \times 3 \times 3 \times 2 = 36$ 種 topic 全部 publish 一次，四個 subscriber 分別只收到 2、6、9、12 則，與預期相符。

補充：若題目格式中最後的「/」代表實際 topic 結尾真的有一個「/」（例如 `house1/roof/brightness/day/`），則 (1)(3) 需寫成 `+/roof/brightness/day/`、`house2/+/+/night/`（多一個空的最後層），(2)(4) 用「#」的寫法不受影響。



```
Subscriber 1
$ mosquitto_sub -v -t '+/roof/brightness/day'
house1/roof/brightness/day house1-roof-brightness-day
house2/roof/brightness/day house2-roof-brightness-day
^C$

Subscriber 2
$ mosquitto_sub -v -t 'house1/firstfloor/#'
house1/firstfloor/temp/day house1-firstfloor-temp-day
house1/firstfloor/temp/night house1-firstfloor-temp-night
house1/firstfloor/humid/day house1-firstfloor-humid-day
house1/firstfloor/humid/night house1-firstfloor-humid-night
house1/firstfloor/brightness/day house1-firstfloor-brightness-day
house1/firstfloor/brightness/night house1-firstfloor-brightness-night
^C$

Subscriber 3
$ mosquitto_sub -v -t 'house2/+/+/night'
house2/firstfloor/temp/night house2-firstfloor-temp-night
house2/firstfloor/humid/night house2-firstfloor-humid-night
house2/firstfloor/brightness/night house2-firstfloor-brightness-night
house2/secondfloor/temp/night house2-secondfloor-temp-night
house2/secondfloor/humid/night house2-secondfloor-humid-night
house2/secondfloor/brightness/night house2-secondfloor-brightness-night
house2/roof/temp/night house2-roof-temp-night
house2/roof/humid/night house2-roof-humid-night
house2/roof/brightness/night house2-roof-brightness-night
^C$

Subscriber 4
$ mosquitto_sub -v -t '+/roof/#'
house1/roof/temp/day house1-roof-temp-day
house1/roof/temp/night house1-roof-temp-night
house1/roof/humid/day house1-roof-humid-day
house1/roof/humid/night house1-roof-humid-night
house1/roof/brightness/day house1-roof-brightness-day
house1/roof/brightness/night house1-roof-brightness-night
house2/roof/temp/day house2-roof-temp-day
house2/roof/temp/night house2-roof-temp-night
house2/roof/humid/day house2-roof-humid-day
house2/roof/humid/night house2-roof-humid-night
house2/roof/brightness/day house2-roof-brightness-day
house2/roof/brightness/night house2-roof-brightness-night
^C$

Publisher
$ for p in house1 house2; do for f in firstfloor secondfloor roof; do for s in temp humid brightness; do for t in day night; do mosquitto_pub -t "$p/$f/$s/$t" -m "$p-$f-$s-$t"; done; done; done; done; echo "published 36 messages"
published 36 messages
$
```

圖 2 Q2 測試：四個 subscriber 同時訂閱，publisher 送出全部 36 種 topic

Q3.

排除匿名使用者截圖(參考簡報範例)，帳號設為學號。

步驟：

用 `sudo mosquitto_passwd -c /etc/mosquitto/passwd <帳號>` 建立帳號（-c 會新建 passwd 檔），建立後 `/etc/mosquitto` 下多了 `passwd`，內容為「帳號:雜湊後的密碼」。

在 `/etc/mosquitto/mosquitto.conf` 加入 `password_file`

/etc/mosquitto/passwd 與 allow_anonymous false , 並 sudo service mosquitto restart。

注意：mosquitto 2.x 若設定檔中沒有任何 listener，會進入 local only mode，此時 allow_anonymous false 不會生效，因此要另外加上 listener 1883。

結果：未帶帳密（匿名）或密碼錯誤的 sub / pub 都被拒絕（Connection Refused: not authorised）；帶正確的 -u / -P 後即可正常訂閱與發佈。

```
Terminal 1 (設定 broker)
$ ls /etc/mosquitto
aclfile.example  ca_certificates  certs  conf.d  mosquitto.conf  pskfile.example  pwfile.example
$ sudo mosquitto_passwd -c /etc/mosquitto/passwd student
Password:
Reenter password:
$ ls /etc/mosquitto
aclfile.example  ca_certificates  certs  conf.d  mosquitto.conf  passwd  pskfile.example  pwfile.example
$ sudo chown mosquitto:mosquitto /etc/mosquitto/passwd
$ sudo cat /etc/mosquitto/passwd
student:$7$101$AnH+fxlD5RCmyOi$WZ08tt013M60txeTnk8niRBaH5/LzxQIyiIv8UHBwA0QD6ceqcuFtX0zD5nbW432VoWcLs0tMPGd
Sjbtlu5FDg==
$ printf '\nlistener 1883\npassword_file /etc/mosquitto/passwd\nallow_anonymous false\n' | sudo tee -a /etc/
mosquitto/mosquitto.conf > /dev/null
$ cat /etc/mosquitto/mosquitto.conf
# Place your local configuration in /etc/mosquitto/conf.d/
#
# A full description of the configuration file is at
# /usr/share/doc/mosquitto/examples/mosquitto.conf.example

#pid_file /run/mosquitto/mosquitto.pid

persistence true
persistence_location /var/lib/mosquitto/

log_dest file /var/log/mosquitto/mosquitto.log

include_dir /etc/mosquitto/conf.d

listener 1883
password_file /etc/mosquitto/passwd
allow_anonymous false
$ sudo service mosquitto restart
* Restarting network daemon mosquitto
...done.
$
```

圖 3 建立帳號、檢視 passwd、修改 mosquitto.conf 並重啟 broker

```
Terminal 2 (subscribe)
$ mosquitto_sub -v -t test
Connection error: Connection Refused: not authorised.
$ mosquitto_sub -v -t test -u student -P 123456
test hello_from_student
^C$

Terminal 3 (publish)
$ mosquitto_pub -t test -m hello
Connection error: Connection Refused: not authorised.
Error: The connection was refused.
$ mosquitto_pub -t test -m hello -u student -P wrongpw
Connection error: Connection Refused: not authorised.
Error: The connection was refused.
$ mosquitto_pub -t test -m hello_from_student -u student -P 123456
$
```

圖 4 匿名 / 錯誤密碼被拒；使用帳號密碼後 subscriber 成功收到訊息

Q4.

Broker : topic、msg	Broker 1 (A) sub 顯示	Broker 2 (B) sub 顯示
A : in、msg1	in msg1	(收不到)
A : out、msg2	out msg2	fromA/out msg2
A : out/msg、msg3	out/msg msg3	fromA/msg msg3
A : /out、msg4	/out msg4	(收不到)
B : out、msg5	fromB/out msg5	out msg5
B : fromB、msg6	fromB/fromB msg6	fromB msg6

Bridge 設定 (設在 Broker A / VM) :

```
connection bridge-01
address <Pi 板 IP>:1883
topic # out 1 out/ fromA/
topic # in 1 fromB/
```

結果說明 :

out 規則 : local-prefix 為 out/、remote-prefix 為 fromA/，A 會在本地訂閱 out/#，符合的訊息轉送給 B，並把開頭的 out/ 換成 fromA/。所以 msg3 (out/msg) 在 B 顯示為 fromA/msg。

msg2 的 topic 是 out : MQTT 的「#」也會匹配它的上一層本身，所以 out 符合 out/# 而被轉送；但它並不是以 "out/" 開頭，沒有前綴可以拿掉，於是只在前面加上 fromA/，B 看到 fromA/out。

msg1 (in) 與 msg4 (/out，第一層是空字串而不是 out) 都不符合 out/#，只留在 A，B 收不到。

in 規則 : 只有 local-prefix fromB/，沒有 remote-prefix，所以 A 在 B 上訂閱「#」，B 上所有訊息都會被帶回 A，並在 topic 前面加上 fromB/ : msg5 → fromB/out、msg6 → fromB/fromB。B 自己的 subscriber 則照原 topic 顯示。

B 上的 fromA/... 訊息不會再被 in 規則帶回 A 形成迴圈，因為 mosquitto 的 bridge 連線會告知遠端 broker 不要把 bridge 自己送出的訊息再回送 (no-local / try_private)。

A 的 subscriber 訂閱「#」，本地 publish 的訊息都會原樣顯示 (out 規則只影

響送往 B 的訊息，不影響 A 本地）。

```
Broker A (VM) 設定
$ printf '\nlistener 1883\nallow_anonymous true\n\nconnection bridge-01\naddress 127.0.0.1:1884\ntopic # out 1 out/ fromA/\ntopic # in 1 fromB/\n' | sudo tee -a /etc/mosquitto/mosquitto.conf > /dev/null
$ tail -8 /etc/mosquitto/mosquitto.conf

listener 1883
allow_anonymous true

connection bridge-01
address 127.0.0.1:1884
topic # out 1 out/ fromA/
topic # in 1 fromB/
$ sudo service mosquitto restart
* Restarting network daemon mosquitto
...done.
$
```

圖 5 在 Broker A 的 mosquitto.conf 加入 bridge 設定並重啟

Broker A (VM) subscribe	Broker B (Pi) subscribe
<pre>\$ mosquitto_sub -v -t '#' in msg1 out msg2 out/msg msg3 /out msg4 fromB/out msg5 fromB/fromB msg6 ^C\$</pre>	<pre>\$ mosquitto_sub -h 127.0.0.1 -p 1884 -v -t '#' fromA/out msg2 fromA/msg msg3 out msg5 fromB msg6 ^C\$</pre>
Broker A (VM) publish	Broker B (Pi) publish
<pre>\$ mosquitto_pub -t in -m msg1 \$ mosquitto_pub -t out -m msg2 \$ mosquitto_pub -t out/msg -m msg3 \$ mosquitto_pub -t /out -m msg4 \$</pre>	<pre>\$ mosquitto_pub -h 127.0.0.1 -p 1884 -t out -m msg5 \$ mosquitto_pub -h 127.0.0.1 -p 1884 -t fromB -m msg6 \$</pre>

圖 6 兩邊 client 皆訂閱 #，依表格順序 publish 的結果

註：此次截圖中的 Broker B 以同一台機器上 port 1884 的 mosquitto 代替 Pi 板（位址 127.0.0.1:1884），bridge 行為與連到 Pi 板相同。

Q5.

在 MQTT bridge configuration 中，QoS level 有哪幾種？分別描述不同 level 的功能。

Ans：QoS 有 0、1、2 三種等級，數字越高傳遞保證越強，但交握次數與負擔也越大。bridge 設定 topic <pattern> <direction> <qos> 中的 qos，是兩個 broker 之間轉送該 topic 時使用的 QoS。

QoS 0 – At most once（最多一次）：送出 PUBLISH 後就不管了，接收端不回覆確認，也不重送。速度最快、負擔最小，但網路不穩時訊息可能遺失。適合頻繁更新、掉一筆也無所謂的感測資料。

QoS 1 – At least once（至少一次）：接收端收到後回 PUBACK；傳送端在收到 PUBACK 前會保存訊息，逾時就重送（DUP 旗標）。保證送達，但可能因重送而收到重複訊息，接收端需能容忍重複。

QoS 2 – Exactly once（剛好一次）：以 PUBLISH → PUBREC → PUBREL → PUBCOMP 四次交握，雙方用 Packet ID 記錄狀態，保證訊息只被處理一次、不遺失也不重複。最可靠但延遲與負擔最大，適合計費、控制指令等不能重

複的訊息。

實際送達的 QoS 會取「發佈端的 QoS」與「訂閱（或 bridge 設定）的 QoS」兩者中較低者。本實驗 bridge 設 QoS 1，代表兩個 broker 之間的轉送保證至少送達一次。

Q6.

在 `mosquitto.conf` 中，對於設定 `cleansession` 的說明如下：

`cleansession [true | false]`

Set the clean session option for this bridge. Setting to *false* (the default), means that all subscriptions on the remote broker are kept in case of the network connection dropping. If set to *true*, all subscriptions and messages on the remote broker will be cleaned up if the connection drops. Note that setting to *true* may cause a large amount of retained messages to be sent each time the bridge reconnects.

If you are using bridges with `cleansession` set to *false* (the default), then you may get unexpected behavior from incoming topics if you change what topics you are subscribing to. This is because the remote broker keeps the subscription for the old topic. If you have this problem, connect your bridge with `cleansession` set to *true*, then reconnect with `cleansession` set to *false* as normal.

請理解 `cleansession` 參數的目的並回答以下問題：

1. 描述 `true` | `false` 分別的作用。
2. 如果 `cleansession` 設置為 `false`，而且你更改訂閱的主題，會出現什麼意外的行為？為什麼會發生這種情況？該怎麼解決？
3. 什麼是“保留消息（retained messages）”，為什麼在 `cleansession` 設置為 `true` 時可能會出現大量的保留消息被發送的情況？

Ans：

(1) `true` / `false` 的作用

`cleansession true`：bridge 以「乾淨的 session」連線到遠端 broker。連線中斷時，遠端 broker 會清除這個 bridge 的所有訂閱以及尚未送出的訊息；重新連線時一切重新開始，bridge 重新訂閱，斷線期間的訊息會遺失。

`cleansession false`（預設）：使用「持久 session」。遠端 broker 以 client id 記住 bridge 的訂閱，斷線時仍保留，並替它暫存斷線期間 QoS 1/2 的訊息；重連後繼續沿用原訂閱並補送這些訊息，不容易掉資料。

(2) cleansession 為 false 時更改訂閱主題的意外行為

現象：修改 bridge 的 topic 設定（例如把 topic sensor/# in 改成 topic light/# in）並重連後，除了新的主題，仍會繼續收到「舊主題」的訊息，而且在設定檔裡已經找不到這條規則。

原因：session 是保存在「遠端 broker」上的。false 時重連會沿用舊 session，遠端 broker 仍記得舊的訂閱；bridge 重連時只會送出新設定中的 SUBSCRIBE，並不知道要去 UNSUBSCRIBE 已經從設定檔刪掉的舊主題，所以新舊訂閱同時存在。

解決：先把 cleansession 設為 true 並重啟讓 bridge 重新連線，遠端就會清掉舊 session（含舊訂閱）；確認之後再改回 false、重啟一次，恢復正常的持久 session。

(3) 保留訊息（retained messages）與大量重送

保留訊息：publish 時設定 retain 旗標（如 mosquitto_pub -r），broker 會為該 topic 保存「最後一筆」保留訊息；之後任何 client 一訂閱到符合的 topic，broker 就立刻把這筆訊息送給它，讓新訂閱者不用等下一次更新就能拿到最新狀態（例如裝置目前的溫度、開關狀態）。

為何 true 時會大量發送：cleansession 為 true 時，每次斷線後遠端都會把訂閱清掉，所以 bridge 每次重連都必須「重新訂閱」。而每一次新的訂閱都會觸發 broker 把所有符合主題的保留訊息整批送出；bridge 常用「#」這類範圍很大的萬用字元，涵蓋的保留訊息可能非常多，網路不穩頻繁重連時，就會一次又一次收到大量重複的保留訊息。false 時訂閱一直保留、不需重新訂閱，就不會發生這種情況。

Q7.心得:

這次實驗實際操作了 MQTT 的 publish / subscribe、帳號驗證與 bridge。Q1 讓我注意到 topic 開頭多一個「/」就等於多了一層空字串，看起來很像的 /a/test 和 a/test 其實完全不同；Q2 則熟悉了「+」只代表單一層、「#」代表之後所有層的差別，用實際發送 36 種 topic 驗證每個 subscriber 收到的筆數，比只在紙上推比較有把握。

設定帳號密碼時遇到一個坑：新版 mosquitto (2.x) 如果設定檔沒有寫 listener，會進入 local only mode，allow_anonymous false 不會生效，匿名使用者照樣可以連線，加上 listener 1883 之後才正確擋下。另外 passwd 檔不存在或 broker 沒有權限讀取時，mosquitto 會直接啟動失敗，client 只看到 Connection refused，要去看 log 才知道原因。

Bridge 的部分最有收穫。out/in 方向搭配 local-prefix、remote-prefix 可以讓兩個

broker 之間只分享部分主題並重新命名，例如 out/msg 到 B 變成 fromA/msg、B 的所有訊息到 A 都加上 fromB/。其中 topic 為 out 的訊息也會被轉送成 fromA/out，是因為「#」也會匹配上一層本身，這是實際測試後才發現的細節。整體而言，MQTT 架構簡單、傳輸量小，透過 broker 讓發佈者與訂閱者互不需要知道對方位址，很適合 IoT 裝置之間的溝通。